



PYTHON BOOTCAMP

www.jomhack.com

CLASSES AND OBJECTS



Classes:

- A template of creating object

Objects:

- Instance of a classes

```
3  # Basic class definition
4  class Person:
5      # Class attribute (shared by all instances)
6      species = "Homo sapiens"
7
8      # Constructor method
9      def __init__(self, name, age):
10         # Instance attributes
11         self.name = name
12         self.age = age
13
14     # Instance method
15     def introduce(self):
16         return f"Hi, I'm {self.name} and I'm {self.age} years old."
17
18     # Method with parameters
19     def have_birthday(self):
20         self.age += 1
21         return f"Happy birthday! {self.name} is now {self.age}."
```

```
23  # Creating objects (instances)
24  person1 = Person("Alice", 25)
25  person2 = Person("Bob", 30)
26
27  # Accessing attributes
28  print(person1.name) # "Alice"
29  print(person1.age)  # 25
30
31  # Calling methods
32  print(person1.introduce())
33  print(person1.have_birthday())
34
35  # Class attributes
36  print(Person.species) # "Homo sapiens"
37  print(person1.species) # "Homo sapiens"
```

CLASSES AND OBJECTS



```
42 class BankAccount:
43     def __init__(self, account_number, owner, balance=0):
44         self.account_number = account_number
45         self.owner = owner
46         self.balance = balance
47         self.transaction_history = []
48
49     def deposit(self, amount):
50         if amount > 0:
51             self.balance += amount
52             self.transaction_history.append(f"Deposited ${amount}")
53             return f"Deposited ${amount}. New balance: ${self.balance}"
54         else:
55             return "Invalid deposit amount"
56
57     def withdraw(self, amount):
58         if amount > 0 and amount <= self.balance:
59             self.balance -= amount
60             self.transaction_history.append(f"Withdrew ${amount}")
61             return f"Withdrew ${amount}. New balance: ${self.balance}"
62         else:
63             return "Invalid withdrawal amount or insufficient funds"
64
65     def get_balance(self):
66         return f"Current balance: ${self.balance}"
67
68     def get_transaction_history(self):
69         return self.transaction_history
```

```
71 # Using the BankAccount class
72 account = BankAccount("12345", "Alice", 1000)
73 print(account.deposit(500))
74 print(account.withdraw(200))
75 print(account.get_balance())
```

CLASSES AND OBJECTS



Exercise:

1. Create a simple game character class with health, attack and heal methods.